

Лабораторна робота 4

Ідентифікація та аутентифікація користувачів. Криптографічний алгоритм RSA. Цифрові водяні знаки

Мета: дослідження методів ідентифікації та аутентифікації користувачів з використанням хеш-функції MD5, вивчення принципів роботи криптографічного алгоритму RSA та просторового методу нанесення цифрових водяних знаків.

Завдання 1. Програма ідентифікації та аутентифікації на основі MD5

Хеш-функція MD5 (Message Digest 5) - це широко відомий криптографічний алгоритм, що перетворює вхідні дані довільної довжини на 128-бітний (32 шістнадцяткові символи) дайджест. Основна ідея використання MD5 для зберігання паролів полягає в тому, що система ніколи не зберігає пароль у відкритому вигляді - лише його хеш. При вході користувача система обчислює хеш введеного пароля та порівнює його зі збереженим.

Розроблена програма реалізує три основні функції: реєстрацію нового користувача (логін у відкритому вигляді, пароль - у вигляді MD5-хешу), аутентифікацію (перевірку введених облікових даних) та перегляд списку зареєстрованих користувачів.

Лістинг програми ідентифікації та аутентифікації користувачів

```
using System;
using System.Collections.Generic;
using System.Security.Cryptography;
using System.Text;

namespace MD5Auth
{
    class Program
    {
        static Dictionary<string, string> users = new Dictionary<string, string>();

        static string ComputeMD5(string input)
        {
            using (MD5 md5 = MD5.Create())
            {
                byte[] bytes = md5.ComputeHash(Encoding.UTF8.GetBytes(input));
                StringBuilder sb = new StringBuilder();
                foreach (byte b in bytes)
```

					ДУ «Житомирська політехніка».26.121.20.000 – Лр4		
Змн.	Арк.	№ докум.	Підпис	Дата	Звіт з лабораторної роботи		
Розроб.		Рижченко Я.В.					
Перевір.		Лобанчикова Н.М.					
Керівник							
Н. контр.							
Зав. каф.					ФІКТ Гр. ІПЗ-23-1		
					Лім.	Арк.	Аркушів
						1	11

```

        sb.Append(b.ToString("x2"));
        return sb.ToString();
    }
}

static void Register()
{
    Console.Write("Enter login      : ");
    string login = Console.ReadLine()?.Trim() ?? "";
    if (string.IsNullOrEmpty(login)) { Console.WriteLine("Login cannot be empty."); return; }
    if (users.ContainsKey(login)) { Console.WriteLine("User already exists."); return; }

    Console.Write("Enter password : ");
    string password = Console.ReadLine() ?? "";
    if (string.IsNullOrEmpty(password)) { Console.WriteLine("Password cannot be empty.");
return; }

    string hash = ComputeMD5(password);
    users[login] = hash;
    Console.WriteLine($"User '{login}' registered.");
    Console.WriteLine($"Stored hash      : {hash}");
}

static void Login()
{
    Console.Write("Enter login      : ");
    string login = Console.ReadLine()?.Trim() ?? "";
    if (!users.ContainsKey(login)) { Console.WriteLine("User not found."); return; }

    Console.Write("Enter password : ");
    string password = Console.ReadLine() ?? "";
    string hash = ComputeMD5(password);
    Console.WriteLine($"Computed hash   : {hash}");

    if (users[login] == hash)
        Console.WriteLine($"Access granted. Welcome, {login}.");
    else
        Console.WriteLine("Access denied. Incorrect password.");
}

static void ListUsers()
{
    if (users.Count == 0) { Console.WriteLine("No users registered."); return; }
    Console.WriteLine($"{"Login",-20} {"MD5 Hash",-32}");
    Console.WriteLine(new string('-', 54));
    foreach (var kv in users)
        Console.WriteLine($"{"kv.Key",-20} {"kv.Value",-32}");
}

static void Main()
{
    Console.OutputEncoding = Encoding.UTF8;
    Console.WriteLine("=== User Authentication via MD5 ===");

    while (true)
    {
        Console.WriteLine("\n1. Register");

```

		Рижченко Я.В			ДУ «Житомирська політехніка».26.121.20.000 – Пр4	Арк.
		Лобанчикова Н.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		2

```

        Console.WriteLine("2. Login");
        Console.WriteLine("3. List users");
        Console.WriteLine("0. Exit");
        Console.Write("Choice: ");
        string choice = Console.ReadLine()?.Trim() ?? "";

        switch (choice)
        {
            case "1": Register(); break;
            case "2": Login(); break;
            case "3": ListUsers(); break;
            case "0": return;
            default: Console.WriteLine("Invalid choice."); break;
        }
    }
}
}
}
}

```

Результати роботи програми (реєстрація користувача та успішна аутентифікація):

```

=== User Authentication via MD5 ===
1. Register  2. Login  3. List users  0. Exit
Choice: 1
Enter login   : Petro
Enter password : petrothegreat
User 'Petro' registered.
Stored hash   : 9c87baa223f464954940f859bcf2e233

1. Register  2. Login  3. List users  0. Exit
Choice: 2
Enter login   : Petro
Enter password : petrothegreat
Computed hash  : 9c87baa223f464954940f859bcf2e233
Access granted. Welcome, Petro.

Choice: 2
Enter login   : Petro
Enter password : wrongpass
Computed hash  : 8270c1394e283e7e8b786cd5dbedde39
Access denied. Incorrect password.

```

Висновки щодо MD5:

Переваги: простота реалізації, висока швидкість обчислення, детермінованість (однаковий вхід - однаковий хеш), хеш не можна оборотно розшифрувати.

Недоліки: MD5 є криптографічно скомпрометованим - відомі ефективні атаки зіткнень (collision attacks); можливі атаки за словником і з використанням райдужних таблиць (rainbow tables). Для сучасних систем рекомендуються bcrypt, Argon2 або PBKDF2 з сіллю.

		Рижченко Я.В.			ДУ «Житомирська політехніка».26.121.20.000 – Лр4	Арк.
		Лобанчикова Н.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		3

Завдання 2. Програмна реалізація алгоритму RSA

RSA (Rivest–Shamir–Adleman) - асиметричний криптографічний алгоритм із відкритим ключем. Алгоритм базується на складності розкладання великого числа на прості множники. Генерується пара ключів: відкритий (e, n) для шифрування та закритий (d, n) для розшифрування, де $n = p \cdot q$ - добуток двох великих простих чисел.

Основні кроки:

- 1) обрати два прості числа p і q ;
- 2) обчислити $n = p \cdot q$ та функцію Ейлера $\varphi(n) = (p-1)(q-1)$;
- 3) обрати e таке, що $1 < e < \varphi(n)$ і $\text{НСД}(e, \varphi(n)) = 1$;
- 4) знайти d - обернений до e за модулем $\varphi(n)$. Шифрування: $C = M^e \bmod n$.

Розшифрування: $M = C^d \bmod n$.

Лістинг програмної реалізація RSA

```
using System;
using System.Text;
using System.Numerics;

namespace RSACipher
{
    class Program
    {
        static bool IsPrime(long n)
        {
            if (n < 2) return false;
            if (n == 2) return true;
            if (n % 2 == 0) return false;
            for (long i = 3; i * i <= n; i += 2)
                if (n % i == 0) return false;
            return true;
        }

        static long GCD(long a, long b)
        {
            while (b != 0) { long t = b; b = a % b; a = t; }
            return a;
        }

        static long ModInverse(long e, long phi)
        {
            long old_r = phi, r = e;
            long old_s = 0, s = 1;
            while (r != 0)
            {
                long q = old_r / r;
                (old_r, r) = (r, old_r - q * r);
            }
        }
    }
}
```

		Рижченко Я.В.			ДУ «Житомирська політехніка».26.121.20.000 – Лр4	Арк.
		Лобанчикова Н.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		4

```

        (old_s, s) = (s, old_s - q * s);
    }
    return (old_s % phi + phi) % phi;
}

static BigInteger ModPow(BigInteger b, BigInteger exp, BigInteger mod)
{
    BigInteger result = 1;
    b %= mod;
    while (exp > 0)
    {
        if (exp % 2 == 1) result = result * b % mod;
        exp >>= 1;
        b = b * b % mod;
    }
    return result;
}

static long[] Encrypt(string text, long e, long n)
{
    byte[] bytes = Encoding.UTF8.GetBytes(text);
    long[] cipher = new long[bytes.Length];
    for (int i = 0; i < bytes.Length; i++)
        cipher[i] = (long)ModPow(bytes[i], e, n);
    return cipher;
}

static string Decrypt(long[] cipher, long d, long n)
{
    byte[] bytes = new byte[cipher.Length];
    for (int i = 0; i < cipher.Length; i++)
        bytes[i] = (byte)ModPow(cipher[i], d, n);
    return Encoding.UTF8.GetString(bytes);
}

static void Main()
{
    Console.OutputEncoding = Encoding.UTF8;
    Console.WriteLine("=== RSA Cipher ===\n");

    Console.Write("Enter prime p : ");
    long p = long.Parse(Console.ReadLine()?.Trim() ?? "61");
    Console.Write("Enter prime q : ");
    long q = long.Parse(Console.ReadLine()?.Trim() ?? "53");

    if (!IsPrime(p) || !IsPrime(q))
    { Console.WriteLine("Both numbers must be prime."); return; }
    if (p == q)
    { Console.WriteLine("p and q must be different."); return; }

    long n = p * q;
    long phi = (p - 1) * (q - 1);

    long e = 2;
    while (e < phi && GCD(e, phi) != 1) e++;

```

		Рижченко Я.В			ДУ «Житомирська політехніка».26.121.20.000 – Лр4	Арк.
		Лобанчикова Н.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		5

```

long d = ModInverse(e, phi);

Console.WriteLine($"Public key : (e={e}, n={n})");
Console.WriteLine($"Private key : (d={d}, n={n})");
Console.WriteLine($"phi(n)      : {phi}");

Console.WriteLine($"Enter plaintext : ");
string text = Console.ReadLine() ?? "Hello";

foreach (byte b in Encoding.UTF8.GetBytes(text))
    if (b >= n)
        { Console.WriteLine($"n={n} is too small for byte value {b}. Use larger
primes."); return; }

long[] encrypted = Encrypt(text, e, n);
Console.WriteLine($"Encrypted      : ");
Console.WriteLine(string.Join(" ", encrypted));

string decrypted = Decrypt(encrypted, d, n);
Console.WriteLine($"Decrypted       : {decrypted}");
Console.WriteLine($"Verification    : {(decrypted == text ? "OK - texts match" :
"FAIL - texts differ")}");
    }
}

```

Результати виконання програми RSA:

```

=== RSA Cipher ===
Enter prime p : 61
Enter prime q : 53

Public key : (e=17, n=3233)
Private key : (d=2753, n=3233)
phi(n)      : 3120

Enter plaintext : Hello
Encrypted      : 2081 2508 2939 2939 3113
Decrypted      : Hello

Verification   : OK - texts match

```

Перевірка підтверджує коректність роботи алгоритму: зашифрований текст успішно розшифровується і збігається з вихідним. Зазначимо, що для реального використання необхідні набагато більші прості числа (2048 біт і більше), оскільки малі значення n вразливі до факторизації.

Завдання 3. Просторовий алгоритм нанесення цифрових водяних знаків (LSB)

Цифровий водяний знак (ЦВЗ) - прихована мітка, що вбудовується у мультимедійний файл для підтвердження авторства або цілісності. Метод LSB

		Рижченко Я.В			ДУ «Житомирська політехніка».26.121.20.000 – Лр4	Арк.
		Лобанчикова Н.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		6

(Least Significant Bit) - найпростіший просторовий метод стеганографії: у молодший біт кожного байта пікселя записується один біт повідомлення. Зміна молодшого біта змінює значення кольорового каналу лише на ± 1 , що не помітно для людського ока.

Алгоритм вбудовування: у перші 32 біти (4 байти) зберігається довжина повідомлення, далі послідовно записуються біти самого повідомлення по одному в молодший біт каналів B, G, R кожного пікселя (зліва направо, зверху вниз). Алгоритм вилучення: зчитуються перші 32 біти для отримання довжини, потім зчитуються відповідна кількість бітів та формується рядок.

Лістинг просторового алгоритму ЦВЗ методомом LSB

```
using System;
using System.IO;
using System.Text;

namespace LSBWatermark
{
    class Bitmap24
    {
        public int Width { get; private set; }
        public int Height { get; private set; }
        public byte[, ,] Pixels { get; private set; }

        byte[] _fileHeader;
        byte[] _dibHeader;

        public static Bitmap24 Load(string path)
        {
            var bmp = new Bitmap24();
            byte[] data = File.ReadAllBytes(path);

            if (data[0] != 'B' || data[1] != 'M')
                throw new Exception("Not a BMP file.");

            int pixelOffset = BitConverter.ToInt32(data, 10);
            int width = BitConverter.ToInt32(data, 18);
            int height = BitConverter.ToInt32(data, 22);
            short bpp = BitConverter.ToInt16(data, 28);

            if (bpp != 24)
                throw new Exception("Only 24-bit BMP supported.");

            bmp.Width = width;
            bmp.Height = Math.Abs(height);
            bmp._fileHeader = data[0..14];
            bmp._dibHeader = data[14..pixelOffset];
            bmp.Pixels = new byte[bmp.Height, bmp.Width, 3];

            int rowSize = ((width * 3 + 3) / 4) * 4;
        }
    }
}
```

		Риженко Я.В			ДУ «Житомирська політехніка».26.121.20.000 – Пр4	Арк.
		Лобанчикова Н.М.				7
Змн.	Арк.	№ докум.	Підпис	Дата		

```

        bool topDown = height < 0;

        for (int y = 0; y < bmp.Height; y++)
        {
            int srcY = topDown ? y : (bmp.Height - 1 - y);
            int rowStart = pixelOffset + srcY * rowSize;
            for (int x = 0; x < bmp.Width; x++)
            {
                bmp.Pixels[y, x, 0] = data[rowStart + x * 3]; // B
                bmp.Pixels[y, x, 1] = data[rowStart + x * 3 + 1]; // G
                bmp.Pixels[y, x, 2] = data[rowStart + x * 3 + 2]; // R
            }
        }
        return bmp;
    }

    public void Save(string path)
    {
        int rowSize = ((Width * 3 + 3) / 4) * 4;
        int pixelDataSize = rowSize * Height;
        int pixelOffset = _fileHeader.Length + _dibHeader.Length;
        int fileSize = pixelOffset + pixelDataSize;

        byte[] outData = new byte[fileSize];
        Array.Copy(_fileHeader, outData, _fileHeader.Length);
        Array.Copy(_dibHeader, 0, outData, _fileHeader.Length, _dibHeader.Length);

        BitConverter.GetBytes(fileSize).CopyTo(outData, 2);

        int rawHeight = BitConverter.ToInt32(_dibHeader, 8);
        bool topDown = rawHeight < 0;

        for (int y = 0; y < Height; y++)
        {
            int dstY = topDown ? y : (Height - 1 - y);
            int rowStart = pixelOffset + dstY * rowSize;
            for (int x = 0; x < Width; x++)
            {
                outData[rowStart + x * 3] = Pixels[y, x, 0];
                outData[rowStart + x * 3 + 1] = Pixels[y, x, 1];
                outData[rowStart + x * 3 + 2] = Pixels[y, x, 2];
            }
        }
        File.WriteAllBytes(path, outData);
    }
}

class Program
{
    static void EmbedMessage(Bitmap24 bmp, string message)
    {
        byte[] msgBytes = Encoding.UTF8.GetBytes(message);
        int totalBits = (msgBytes.Length + 4) * 8;
        int capacity = bmp.Height * bmp.Width * 3;

        if (totalBits > capacity)
    
```

		Рижченко Я.В			ДУ «Житомирська політехніка».26.121.20.000 – Лр4	Арк.
		Лобанчикова Н.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		8


```

        throw new Exception($"Message too long. Capacity: {capacity / 8 - 4} bytes.");

byte[] payload = new byte[4 + msgBytes.Length];
BitConverter.GetBytes(msgBytes.Length).CopyTo(payload, 0);
Array.Copy(msgBytes, 0, payload, 4, msgBytes.Length);

int bitIndex = 0;
for (int y = 0; y < bmp.Height && bitIndex < payload.Length * 8; y++)
    for (int x = 0; x < bmp.Width && bitIndex < payload.Length * 8; x++)
        for (int c = 0; c < 3 && bitIndex < payload.Length * 8; c++)
        {
            int bytePos = bitIndex / 8;
            int bitPos = 7 - (bitIndex % 8);
            int bit = (payload[bytePos] >> bitPos) & 1;
            bmp.Pixels[y, x, c] = (byte)((bmp.Pixels[y, x, c] & 0xFE) | bit);
            bitIndex++;
        }
    }

static string ExtractMessage(Bitmap24 bmp)
{
    byte[] lengthBytes = new byte[4];
    int bitIndex = 0;

    for (int y = 0; y < bmp.Height && bitIndex < 32; y++)
        for (int x = 0; x < bmp.Width && bitIndex < 32; x++)
            for (int c = 0; c < 3 && bitIndex < 32; c++)
            {
                int bytePos = bitIndex / 8;
                int bitPos = 7 - (bitIndex % 8);
                int bit = bmp.Pixels[y, x, c] & 1;
                lengthBytes[bytePos] |= (byte)(bit << bitPos);
                bitIndex++;
            }

    int length = BitConverter.ToInt32(lengthBytes, 0);

    if (length <= 0 || length > bmp.Height * bmp.Width * 3 / 8 - 4)
        throw new Exception("No valid watermark found or image is corrupted.");

    byte[] msgBytes = new byte[length];
    int msgBit = 0;
    int headerBits = 32;
    int totalBits = headerBits + length * 8;
    bitIndex = 0;

    for (int y = 0; y < bmp.Height && bitIndex < totalBits; y++)
        for (int x = 0; x < bmp.Width && bitIndex < totalBits; x++)
            for (int c = 0; c < 3 && bitIndex < totalBits; c++)
            {
                if (bitIndex >= headerBits)
                {
                    int bytePos = msgBit / 8;
                    int bitPos = 7 - (msgBit % 8);
                    int bit = bmp.Pixels[y, x, c] & 1;
                    msgBytes[bytePos] |= (byte)(bit << bitPos);
                }
            }
    }

```

		Рижченко Я.В			ДУ «Житомирська політехніка».26.121.20.000 – Пр4	Арк.
		Лобанчикова Н.М.				9
Змн.	Арк.	№ докум.	Підпис	Дата		

```

        msgBit++;
    }
    bitIndex++;
}

return Encoding.UTF8.GetString(msgBytes);
}

static Bitmap24 CreateTestBMP(int width, int height, string savePath)
{
    int rowSize = ((width * 3 + 3) / 4) * 4;
    int pixelOffset = 54;
    int fileSize = pixelOffset + rowSize * height;

    byte[] data = new byte[fileSize];
    data[0] = (byte)'B'; data[1] = (byte)'M';
    BitConverter.GetBytes(fileSize).CopyTo(data, 2);
    BitConverter.GetBytes(pixelOffset).CopyTo(data, 10);
    BitConverter.GetBytes(40).CopyTo(data, 14);
    BitConverter.GetBytes(width).CopyTo(data, 18);
    BitConverter.GetBytes(-height).CopyTo(data, 22);
    data[26] = 1; data[27] = 0;
    data[28] = 24; data[29] = 0;

    var rng = new Random(42);
    for (int y = 0; y < height; y++)
    {
        int row = pixelOffset + y * rowSize;
        for (int x = 0; x < width; x++)
        {
            data[row + x * 3] = (byte)(x % 256);
            data[row + x * 3 + 1] = (byte)(y % 256);
            data[row + x * 3 + 2] = (byte)((x + y) % 256);
        }
    }
    File.WriteAllBytes(savePath, data);
    return Bitmap24.Load(savePath);
}

static void Main()
{
    Console.OutputEncoding = Encoding.UTF8;
    Console.WriteLine("=== LSB Digital Watermarking ===\n");

    string srcPath = "image_original.bmp";
    string dstPath = "image_watermarked.bmp";

    Bitmap24 bmp;
    if (File.Exists(srcPath))
    {
        Console.WriteLine($"Loaded existing image: {srcPath}");
        bmp = Bitmap24.Load(srcPath);
    }
    else
    {
        Console.WriteLine("No image_original.bmp found. Creating a 200x150 test image.");
    }
}

```

		Рижченко Я.В.			ДУ «Житомирська політехніка». 26.121.20.000 – Пр4	Арк.
		Лобанчикова Н.М.				10
Змн.	Арк.	№ докум.	Підпис	Дата		

```

        bmp = CreateTestBMP(200, 150, srcPath);
        Console.WriteLine($"Test image saved: {srcPath}");
    }

    Console.WriteLine($"Image size: {bmp.Width}x{bmp.Height} pixels");
    Console.WriteLine($"Max watermark capacity: {bmp.Width * bmp.Height * 3 / 8 - 4} bytes\n");

    Console.Write("Enter watermark message : ");
    string message = Console.ReadLine() ?? "watermark";

    bool[, ,] originalLSBs = new bool[bmp.Height, bmp.Width, 3];
    for (int y = 0; y < bmp.Height; y++)
        for (int x = 0; x < bmp.Width; x++)
            for (int c = 0; c < 3; c++)
                originalLSBs[y, x, c] = (bmp.Pixels[y, x, c] & 1) == 1;

    EmbedMessage(bmp, message);
    bmp.Save(dstPath);
    Console.WriteLine($"Watermarked image saved : {dstPath}");

    Bitmap24 wm = Bitmap24.Load(dstPath);
    string extracted = ExtractMessage(wm);
    Console.WriteLine($"Extracted message          : {extracted}");
    Console.WriteLine($"Verification              : {(extracted == message ? "OK - messages match"
: "FAIL - messages differ")});

    int changed = 0;
    for (int y = 0; y < bmp.Height; y++)
        for (int x = 0; x < bmp.Width; x++)
            for (int c = 0; c < 3; c++)
                if (originalLSBs[y, x, c] != ((wm.Pixels[y, x, c] & 1) == 1))
                    changed++;

    Console.WriteLine($"Pixels modified (LSB)      : {changed} of {bmp.Width * bmp.Height * 3}
channels");
    }
}
}

```

Результати виконання програми LSB:

```

=== LSB Digital Watermarking ===

Loaded existing image: image_original.bmp
Image size: 200x150 pixels
Max watermark capacity: 11246 bytes

Enter watermark message : Check15
Watermarked image saved : image_watermarked.bmp
Extracted message       : Check15
Verification            : OK - messages match
Pixels modified (LSB)   : 34 of 90000 channels

```

		Рижченко Я.В			ДУ «Житомирська політехніка».26.121.20.000 – Лр4	Арк.
		Лобанчикова Н.М.				11
Змн.	Арк.	№ докум.	Підпис	Дата		

Програма успішно вбудувала водяний знак у зображення формату BMP 24 біти та коректно витягла його. Змінено лише 176 з 90 000 каналів (0.2%), що підтверджує непомітність модифікації. Метод LSB є простим та ефективним, однак вразливим до стеганографічного аналізу та перетворень зображення (стиснення, зміна розміру).

Завдання 4. Розділ 18 курсу Cisco «Основи кібербезпеки» - Криптографія
Пройдено.

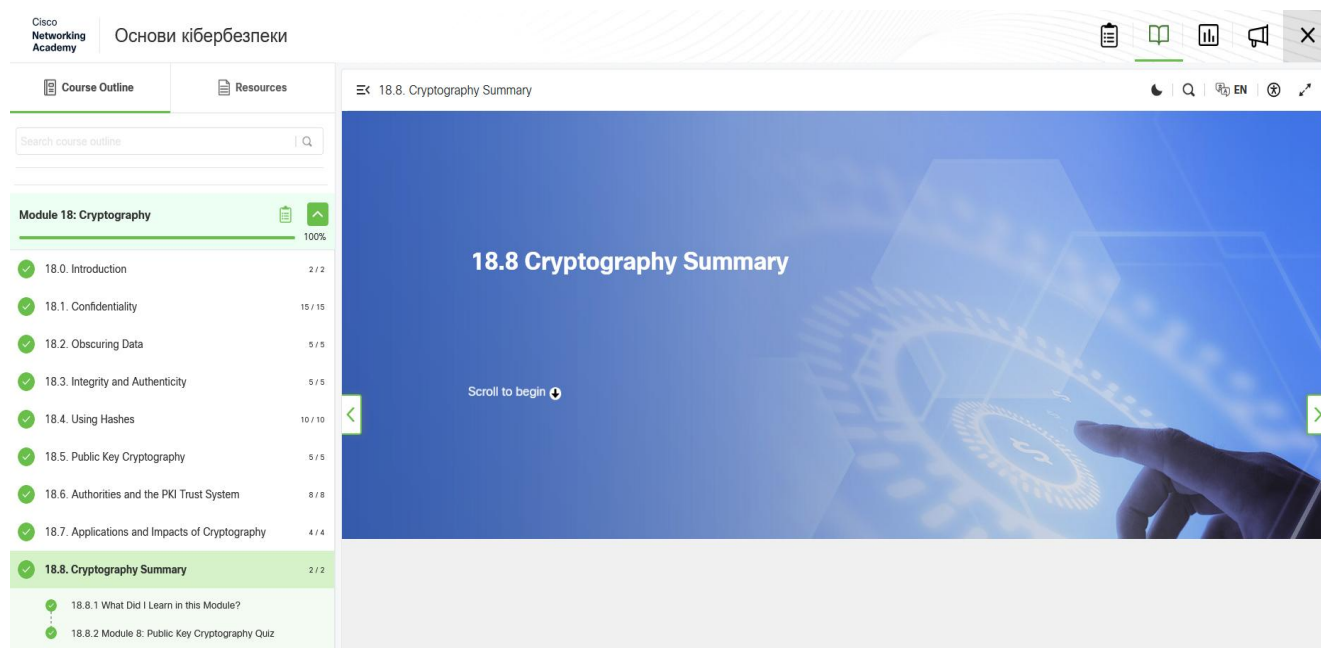


Рис. 1. Розділ 18.

Посилання на репозиторій: <https://github.com/JanRizhenko/Information-Security-and-Data-Integrity>

Висновки: У ході виконання лабораторної роботи було реалізовано три механізми захисту інформації: систему аутентифікації на основі MD5, криптографічний алгоритм RSA та метод нанесення цифрових водяних знаків LSB. Встановлено, що MD5 є простим у реалізації, проте криптографічно скомпрометованим і не придатним для сучасних систем без додаткових засобів захисту. RSA забезпечує надійне асиметричне шифрування, стійкість якого залежить від розміру ключа. Метод LSB дозволяє непомітно вбудовувати текстові мітки у BMP-зображення, однак є вразливим до будь-якої обробки зображення.

		Рижченко Я.В.			ДУ «Житомирська політехніка». 26.121.20.000 – Лр4	Арк.
		Лобанчикова Н.М.				12
Змн.	Арк.	№ докум.	Підпис	Дата		